

INHERITED FROM Helix Constitution

This module is a submodule of a consuming project that includes the Helix Constitution submodule at the parent's `constitution/` path. All rules in `constitution/CLAUDE.md` and the `constitution/Constitution.md` it references (universal anti-bluff covenant Â§11.4, no-guessing mandate Â§11.4.6, credentials-handling mandate Â§11.4.10, host-session safety Â§12, data safety Â§9, mutation- paired gates Â§1.1) apply unconditionally to every change landed here. The module-specific rules below extend them â€” they never weaken any universal clause.

When this file disagrees with the constitution submodule, the constitution wins. Locate the constitution submodule from any arbitrary nested depth using its `find_constitution.sh` helper.

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution>

The project Constitution

Inherits from: `HelixConstitution/Constitution.md` â€” all universal clauses in that document apply unconditionally. Rules below extend or tighten them; they MUST NOT weaken any inherited clause.

The project Constitution

Version: 1.0.0 **Effective Date:** 2026-04-30 **Scope:** This Constitution applies to the project and ALL its submodules **Authority:** Cascaded from the parent project root governance with project-specific addenda

Preamble

The project is an enterprise-grade distributed AI development platform. This Constitution establishes the non-negotiable rules that govern all development, testing, deployment, and maintenance activities within the project. Every contributor, agent, and automated process MUST adhere to these rules. No exceptions.

CONST-001: No CI/CD Pipelines (Permanent)

No `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, `.travis.yml`, `.circleci/`, or any automated pipeline. No Git hooks. All builds and tests run manually or via `Makefile`/script targets.

Rationale: Manual execution ensures human oversight and prevents automated propagation of bluffs.

CONST-002: No Mocks in Production (Permanent)

CONST-002a: Production Code

Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. All production code is fully functional with real integrations.

CONST-002b: Test Code

Mocks/stubs/fakes MAY be used ONLY in unit tests (files ending `_test.go` run under `go test -short`).

Rationale: Production bluffs have repeatedly been discovered where features appeared implemented but were non-functional.

CONST-003: No HTTPS for Git (Permanent)

SSH URLs only (`git@github.com:â€¦`, `git@gitlab.com:â€¦`, etc.) for clones, fetches, pushes, and submodule updates. SSH keys are configured on every service.

CONST-004: No Manual Container Commands (Permanent)

Container orchestration is owned by the project's binary/orchestrator (e.g., `make build` or `./bin/<app>`). Direct `docker/podman start|stop|rm` and `docker-compose up|down` are prohibited as workflows.

CONST-005: 100% Real Data for Non-Unit Tests

Beyond unit tests, all components MUST use actual API calls, real databases, live services. No simulated success. Fallback chains tested with actual failures.

Verification: Every integration/E2E test MUST connect to real services or skip (not fail) if unavailable.

CONST-006: Challenge Coverage (Permanent)

Every component MUST have Challenge scripts (`./challenges/scripts/`) validating real-life use cases. No false success â€” validate actual behavior, not return codes.

CONST-007: Health & Observability

Every service MUST expose health endpoints. Circuit breakers for all external dependencies. Prometheus / OpenTelemetry integration where applicable.

CONST-008: Documentation & Quality

Update CLAUDE.md, AGENTS.md, and relevant docs alongside code changes. Pass language-appropriate format/lint/security gates. Conventional Commits: `<type>(<scope>): <description>`.

CONST-009: Validation Before Release

Pass the project's full validation suite (`make ci-validate-all-equivalent`) plus all challenges (`./challenges/scripts/run_all_challenges.sh`).

CONST-010: Comprehensive Verification

Every fix MUST be verified from all angles: runtime testing (actual HTTP requests / real CLI invocations), compile verification, code structure checks, dependency existence checks, backward compatibility, and no false positives. Grep-only validation is NEVER sufficient.

CONST-011: Resource Limits for Tests & Challenges

ALL test and challenge execution MUST be strictly limited to 30-40% of host system resources. Use `GOMAXPROCS=2, nice -n 19, ionice -c 3, -p 1` for `go test`. Container limits required.

CONST-012: Bugfix Documentation

All bug fixes MUST be documented in `docs/issues/fixed/BUGFIXES.md` with root cause analysis, affected files, fix description, and a link to the verification test/challenge.

CONST-013: Real Infrastructure for All Non-Unit Tests

Mocks/fakes/stubs/placeholders MAY be used ONLY in unit tests. ALL other test types "integration, E2E, functional, security, stress, chaos, challenge, benchmark, runtime verification" MUST execute against REAL running systems with REAL containers, REAL databases, REAL services, and REAL HTTP calls.

CONST-014: Reproduction-Before-Fix (Mandatory)

Every reported error, defect, or unexpected behavior MUST be reproduced by a Challenge script BEFORE any fix is attempted. Sequence: 1. Write the Challenge first 2. Run it; confirm fail (it reproduces the bug) 3. Then write the fix 4. Re-run; confirm pass 5. Commit Challenge + fix together

The Challenge becomes the regression guard for that bug forever.

CONST-015: Concurrent-Safe Containers

Any struct field that is a mutable collection (map, slice) accessed concurrently **MUST** use thread-safe primitives. Bare `sync.Mutex + map/slice` combinations are prohibited for new code.

CONST-016: Definition of Done (Universal)

A change is NOT done because code compiles and tests pass. “Done” requires pasted terminal output from a real run.

- **No self-certification:** Words like *verified*, *tested*, *working*, *complete*, *fixed*, *passing* are forbidden in commits/PRs/replies unless accompanied by pasted output from a command that ran in that session.
 - **Demo before code:** Every task begins by writing the runnable acceptance demo
 - **Real system, every time:** Demos run against real artifacts
 - **Skips are loud:** `t.Skip` without a trailing `SKIP-OK: #<ticket>` comment breaks validation
-

CONST-019: Container Up % Healthy

Container Up status does NOT mean the application is healthy. Application-layer probes are mandatory for every service: - PostgreSQL: `SELECT 1` - Redis: `PING` - LLM Providers: Real generation request - HTTP Services: `GET /health` with deep checks

CONST-020: Provider Fallback Chain Reality

Every LLM provider fallback chain **MUST** be tested with actual failures. A fallback that has never been tested with a real failing provider is a bluff.

CONST-021: No Mocks Above Unit Build Target

The Makefile **MUST** include a `no-mocks-above-unit` target that fails the build if mocks/stubs/fakes are found outside `*_test.go` files.

CONST-022: Submodule Governance Propagation

Every submodule **MUST** either: 1. Have its own `Constitution.md`, `CLAUDE.md`, and `AGENTS.md`, OR 2. Have a symlink to the parent repository’s governance files, OR 3. Have a reference comment in its `README` pointing to parent governance

No submodule is exempt from these rules.

CONST-023: Docker Health Checks Mandatory

Every Dockerfile MUST include:

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1
```

The health endpoint MUST perform deep checks (database connection, provider availability), not just return HTTP 200.

CONST-024: Version Pinning

All dependencies MUST be pinned to specific versions in `go.mod`. No `latest`, no floating tags. Renovate or Dependabot (manual review only – see CONST-001) may propose updates.

CONST-025: Secret Management

NO secrets in code. EVER. Secrets via: - Environment variables (production) - `.env` files (development, in `.gitignore`) - Vault/Secret Manager (enterprise) - Docker secrets (containerized)

`go mod tidy` MUST NOT add secret-scanning bypasses.

CONST-026: Minimal Privilege Containers

Containers run as non-root. Every Dockerfile:

```
RUN adduser -D -u 1001 helixcode
USER helixcode
```

CONST-027: Network Isolation

Container orchestration MUST use internal networks. Services communicate via named hosts, not exposed ports where possible.

CONST-028: Backup Before Destructive Operations

Every file editing tool MUST create backups before modification. The backup MUST be restorable.

CONST-029: Input Validation at All Boundaries

Every public function MUST validate inputs. No trust of caller-provided data. SQL injection, path traversal, command injection MUST be impossible by design.

CONST-030: Graceful Degradation

When external services are unavailable, the system MUST degrade gracefully: - Return partial results where possible - Queue operations for retry - Inform user of degraded state - NEVER crash or hang indefinitely

CONST-031: Audit Trail

Every significant operation MUST be logged with: - Timestamp - User identity - Operation type - Success/failure status - Resource affected

Log retention: 90 days minimum.

CONST-032: Emergency Stop

Every long-running or distributed operation MUST support cancellation via `context.Context`. Users MUST be able to interrupt any operation.

CONST-033: Data Integrity

Database writes MUST be transactional. Partial writes MUST be rolled back. Consistency checks MUST run periodically.

CONST-034: API Stability

Public APIs maintain backward compatibility within major versions. Deprecation requires: - 6-month notice - Migration guide - Compatibility shim

CONST-036: Propagation to Submodules

This Constitution, along with `CLAUDE.md` and `AGENTS.md`, MUST be propagated to ALL submodules. Each submodule's governance MUST reference this parent Constitution. Changes to this Constitution MUST trigger review of all submodule governance files.

CONST-037: LLMsVerifier Single Source of Truth Mandate

Rule: LLMsVerifier SHALL BE the sole authoritative source for: 1. All model metadata (names, IDs, context windows, capabilities) 2. All provider metadata (endpoints, auth types, supported models) 3. All verification status (verified, partial, failed, pending) 4. All scoring data (overall scores, capability scores, tier rankings) 5. All rate-limit and cooldown state

Prohibition: NO hardcoded model lists, NO hardcoded provider lists, NO simulated model discovery. Any code path that presents a model or provider listing to a user MUST fetch that listing from the LLMsVerifier subsystem or its cached replica.

Anti-Bluff Verification: - The challenge script `challenges/scripts/verifier_hardcode_check.sh` MUST scan all Go source files for hardcoded model arrays. - Any `[]string{"gpt-4", "claude-3"}` or equivalent literal in production code is a constitutional violation. - The only permitted hardcoded data is the LLMsVerifier service endpoint URL and the list of verification test types.

Enforcement: `make test-complete` MUST include a test that asserts `ModelManager.GetAvailableModels()` returns at least as many models as the verifier's database contains for configured providers. A test that passes while the CLI shows a hardcoded list is a TEST BLUFF and violates CONST-035.

CONST-038: Model Provider Anti-Bluff Guarantee

Rule: Every model displayed to an end user MUST have been verified by LLMsVerifier within the last `verification_timeout` period (default: 24h). Models older than this MUST display a "stale" indicator and be deprioritized.

Prohibition Against Test Bluffing: - A unit test that mocks the verifier client and asserts `GetAvailableModels()` returns 3 models DOES NOT satisfy this rule. - An integration test that starts the verifier server, performs real provider discovery, and confirms the model count matches the actual provider API response DOES satisfy this rule. - The Makefile target `make test-verifier-integration` MUST exist and MUST run without mocks.

The "Tests Pass But Features Don't Work" Guarantee:

NO TEST MAY PASS UNLESS THE FEATURE IT TESTS IS DEMONSTRABLY USABLE BY AN END USER IN THE SAME BUILD.

- If `TestModelList` passes but `helixcode --list-models` shows hardcoded data, the test is a BLUFF.
- If `TestProviderHealth` passes but the health endpoint returns `200 OK` for a provider that is actually down, the test is a BLUFF.
- If `TestLLMGeneration` passes but `--prompt "hello"` returns a simulated string, the test is a BLUFF.
- Bluff tests MUST be rewritten or deleted. There is no "grandfather" exception.

Evidence Standard: Every test that claims to verify model/provider functionality MUST: 1. Call a real API endpoint or a real verifier database 2. Assert on response content that could only come from that real source 3. Include a test that runs the CLI binary with `--list-models` and checks output against verifier data

CONST-039: Real-Time Model Status Accuracy

Rule: Model status (available, rate-limited, cooldown, offline, deprecated) displayed to users MUST reflect the actual state as known by LLMsVerifier within `max_staleness` seconds (default: 60s).

Polling vs. Push: - If WebSocket/SSE push is unavailable, the system MUST poll LLMsVerifier at most every `status_poll_interval` (default: 30s). - The TUI MUST display a "last updated" timestamp with every model listing. - Models in "cooldown" or "rate-limited" state MUST show the estimated recovery time if known.

Accuracy Verification: - Challenge script `challenges/scripts/model_status_accuracy_challenge.sh` MUST: 1. Artificially rate-limit a provider by exhausting its quota 2. Wait for the status to propagate to the verifier 3. Check that `helixcode --list-models` shows the rate-limited status within 60s 4. Check that `SelectOptimalModel()` no longer selects the rate-limited model

Prohibition: Status indicators that are "always green" or that lag >60s behind reality violate this rule.

CONST-040: All Providers and Models Integration Mandate

Rule: The project MUST integrate with ALL providers and models that LLMsVerifier supports, subject only to: 1. The provider being explicitly disabled in configuration (`enabled: false`) 2. The API key being absent and the provider requiring one 3. The provider being marked deprecated in the verifier database

Minimum Provider Set (SHALL NOT be reduced without constitutional amendment): | Provider | Auth Type | Required Env Var | | OpenAI | API Key | OPENAI_API_KEY | | Anthropic | API Key / OAuth | ANTHROPIC_API_KEY | | Gemini | API Key | GEMINI_API_KEY | | DeepSeek | API Key | DEEPSEEK_API_KEY | | Groq | API Key | GROQ_API_KEY | | Mistral | API Key | MISTRAL_API_KEY | | xAI | API Key | XAI_API_KEY | | OpenRouter | API Key | OPENROUTER_API_KEY | | Ollama | Local | None (auto-detect) | | Llama.cpp | Local | None (auto-detect) |

Integration Requirement: For every provider in the minimum set: - There MUST be a provider adapter file in `internal/llm/` or `internal/verifier/adapters/` - There MUST be a `*_test.go` file with real API tests (skipped only if `HELIX_SKIP_LIVE_PROVIDER_TESTS` is set) - There MUST be a challenge script in `challenges/scripts/` - The model listing MUST include models from this provider when the provider is enabled

CONST-041: MCP / LSP / ACP / Embedding / RAG / Skills / Plugins Integration Mandate

Rule: LLMsVerifier integration SHALL extend beyond basic model listing to cover ALL capability dimensions:

1. **MCP (Model Context Protocol):** The verifier MUST report which models support MCP tool calling. The project's MCP subsystem MUST consult verifier capability flags before selecting a model for tool-use tasks.
2. **LSP (Language Server Protocol):** The verifier MUST report code-analysis capabilities. Models without `code_analysis` capability MUST NOT be selected for refactoring or debugging tasks.
3. **ACP (Agent Capability Protocol):** The verifier MUST report multi-agent coordination support. Models with `supports_parallel_tool_use` MUST be preferred for ACP workflows.
4. **Embedding:** The verifier MUST report `supports_embeddings` for each model. The `CogneeConfig` embedding model selection MUST be verifier-aware.
5. **RAG (Retrieval-Augmented Generation):** The verifier MUST report context-window sizes. RAG chunking strategies MUST adapt to the selected model's `context_window_tokens` as reported by the verifier.
6. **Skills / Plugins:** The verifier MUST track plugin compatibility. Models flagged `plugin_compatible` MUST be used when skill/plugin execution is required.

Capability Checklist (MUST be verified by challenge): - [] MCP tool calling verified for at least 3 providers - [] LSP code-analysis verified for at least 3 providers - [] ACP parallel tool use verified for at least 2 providers - [] Embedding generation verified for at least 2 providers - [] RAG context-window adaptation verified - [] Skills/plugin execution verified for at least 2 providers

Prohibition: Capability flags MUST NOT be hardcoded. The `Provider.GetCapabilities()` method MUST return data sourced from the verifier's `VerificationResult` fields.

Article XII "Repository Safety"

§12.1 (CONST-042) "No-Secret-Leak"

No API key, token, password, certificate, or other credential may be committed to any repository owned by HelixDevelopment or vasic-digital, transitively or otherwise. All secrets live in `.env` files (mode 0600) listed in `.gitignore`. Any leak "to git, logs, build artefacts, screenshots, or external services" is a release blocker until rotated and post-mortemed.

Operational requirements: - Every repo must have `.env`, `.env.local`, `.env.*` (with `!.env.example` exception), `*.pem`, `*.key`, `*.crt`, `id_rsa*` in `.gitignore`. - `scripts/scan-secrets.sh` (or equivalent) must run before every push; failing it blocks the push. - API keys for development are sourced from the canonical `../helix_agent/.env`

(mode 0600, never under git) and copied “ never symlinked, never committed ” into per-repo .env files.

Cascade requirement: This article must appear verbatim in every owned-by-us repository’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Owned-by-us repos are listed in scripts/owned-repos.txt (or, until that file exists, the meta-repo propagate-governance.sh script’s submodule walk excluding third-party trees).

Â§12.2 (CONST-043) “ No-Force-Push

No force push, force-with-lease push, history rewrite, branch deletion of main/master, or upstream-overwriting operation may be performed without explicit, in-conversation user approval given for that specific operation. Authorization for one push does not extend to subsequent pushes. Bypassing hooks (--no-verify), signature verification (--no-gpg-sign), or protected-branch rules also requires explicit approval. This applies to every repository in the HelixDevelopment / vasic-digital stack.

Operational requirements: - Local pre-push hook at scripts/git-hooks/pre-push (installed by scripts/install-git-hooks.sh) must reject --force / --force-with-lease unless HELIX_FORCE_PUSH_APPROVED=1 is set. - The hook is a courtesy gate; this constitutional clause is the actual contract. - Regular non-force pushes of new commits to existing branches on already-configured remotes are PERMITTED without per-push approval, scoped to a programme/conversation in which the user has authorised the cadence.

Cascade requirement: Same as Â§12.1 “ verbatim, every owned-by-us repo’s three governance files.